7579번 - 앱 성공

☆

골드 III

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
1 초	128 MB	48034	16374	11900	34.574%

문제

우리는 스마트폰을 사용하면서 여러 가지 앱(App)을 실행하게 된다. 대개의 경우 화면에 보이는 '실행 중'인 앱은 하나뿐이지만 보이지 않는 상태로 많은 앱이 '활성화'되어 있다. 앱들이 활성화 되어 있다는 것은 화면에 보이지 않더라도 메인 메모리에 직전의 상태가 기록되어 있는 것을 말한다. 현재 실행 중이 아니더라도 이렇게 메모리에 남겨두는 이유는 사용자가 이전에 실행하던 앱을 다시 불러올 때에 직전의 상태를 메인 메모리로부터 읽어 들여 실행 준비를 빠르게 마치기 위해서이다.

하지만 스마트폰의 메모리는 제한적이기 때문에 한번이라도 실행했던 모든 앱을 활성화된 채로 메인 메모리에 남겨두다 보면 메모리 부족 상태가 오기 쉽다. 새로운 앱을 실행시키기 위해 필요한 메모리가 부족해지면 스마트폰의 운영체제는 활성화 되어 있는 앱들 중 몇 개를 선택하여 메모리로부터 삭제하는 수밖에 없다. 이러한 과정을 앱의 '비활성화'라고 한다.

메모리 부족 상황에서 활성화 되어 있는 앱들을 무작위로 필요한 메모리만큼 비활성화 하는 것은 좋은 방법이 아니다. 비활성화된 앱들을 재실행할 경우 그만큼 시간이 더 필요하기 때문이다. 여러분은 이러한 앱의 비활성화 문제를 스마트하게 해결하기 위한 프로그램을 작성해야 한다

현재 N개의 앱, $A_1, \ldots, A_N$이 활성화 되어 있다고 가정하자. 이들 앱 A_i는 각각 m_i 바이트만큼의 메모리를 사용하고 있다. 또한, 앱 A_i를 비활성화한 후에 다시 실행하고자 할 경우, 추가적으로 들어가는 비용(시간 등)을 수치화 한 것을 c_i 라고 하자. 이러한 상황에서 사용자가 새로운 앱 B를 실행하고자 하여, 추가로 M 바이트의 메모리가 필요하다고 하자. 즉, 현재 활성화 되어 있는 앱 $A_1, \ldots, A_N$ 중에서 몇 개를 비활성화 하여 M 바이트 이상의 메모리를 추가로 확보해야 하는 것이다. 여러분은 그 중에서 비활성화 했을 경우의 비용 c_i의 합을 최소화하여 필요한 메모리 M 바이트를 확보하는 방법을 찾아야 한다.

입력

입력은 3줄로 이루어져 있다. 첫 줄에는 정수 N과 M이 공백문자로 구분되어 주어지며, 둘째 줄과 셋째 줄에는 각각 N개의 정수가 공백문자로 구분되어 주어진다. 둘째 줄의 N개의 정수는 현재 활성화 되어 있는 앱 $A_1, \ldots, A_N$이 사용 중인 메모리의 바이트 수인 $m_1, \ldots, m_N$을 의미하며, 셋째 줄의 정수는 각 앱을 비활성화 했을 경우의 비용 $c_1, \ldots, c_N$을 의미한다

단, $1 \le N \le 100$, $1 \le M \le 10{,}000{,}000$이며, $1 \le m_1, \ldots, m_N \le 10{,}000{,}000$을 만족한다. 또한, $0 \le c_1, \ldots, c_N \le 100$이고, $M \le m_1 + m_2 + \ldots + m_N$이다.

출력

필요한 메모리 M 바이트를 확보하기 위한 앱 비활성화의 최소의 비용을 계산하여 한 줄에 출력해야 한다.

예제 입력 1 복사

```
5 60
30 10 20 35 40
3 0 3 5 4
```

예제 출력 1 복사

```
6
```

출처

Olympiad (/category/2) > 한국정보올림피아드 (/category/55) > 한국정보올림피아드시.도지역본선 (/category/57) > 지역본선 2013 (/category/214) > 고등부 (/category/detail/910) 4번

- 잘못된 데이터를 찾은 사람: tncks0121 (/user/tncks0121)
- 데이터를 추가한 사람: 20191647 (/user/20191647), degurii (/user/degurii), gojib2002 (/user/gojib2002), shjohw12 (/user/shjohw12), sjyrsh0712 (/user/sjyrsh0712)
- 빠진 조건을 찾은 사람: yclock (/user/yclock)

알고리즘 분류

- 다이나믹 프로그래밍 (/problem/tag/25)
- 배낭 문제 (/problem/tag/148)

메모